

RNG API

Introduction

This document describes different ways of using *RNG* (*Random Number Generator*).

The key words `MUST`, `MUST NOT`, `REQUIRED`, `SHALL`, `SHALL NOT`, `SHOULD`, `SHOULD NOT`, `RECOMMENDED`, `MAY`, and `OPTIONAL` in this document are to be interpreted as described in [RFC 2119](#).

Algorithms

The module can use one of the selected pseudo RNG algorithms:

- `issac` . Default algorithm. This algorithm is certified
- `mersenne-twister`

Issac algorithm

RNG uses certified [ISAAC](#) algorithm to generate numbers.

ISAAC is a [CSPRNG](#) designed by [Robert J. Jenkins Jr.](#) in 1996 and based on RC4. It is designed to be fast and secure. *isaac.js* is fully compatible with the original * 32-bit integer arithmetic* implementations of ISAAC.

ISAAC can generate cryptographically secure pseudorandom numbers from an input but do not provide any entropy source. It's the responsibility of the user to seed ISAAC with a strong entropy source. The Isaac RNG algorithm is known for its high-quality randomness and is designed to be robust and secure. It uses a large internal state and a variety of bitwise operations and permutations to achieve its properties. The algorithm incorporates features such as mixing, avalanche effects, and a well-designed update function to ensure a long period and strong statistical properties in the generated random sequences.

Scaling & Interval

The generated 32-bit integer is converted into float in range of `[0, 1)` by multipling it by `2-32` .

Mapping

RNG returns just a single float, mapping it to a specific range in unbiased fashion is done with a `randomInteger` function.

Period

The Isaac RNG algorithm is designed to have a very long period, meaning it can produce a large number of distinct random sequences before cycling.

Seeding

Seed is generated based on three independent parameters:

- random number from internal node.js crypto library
- process ID
- free memory
- timestamp

There is no periodic re-seeding.

RNG monitoring

Certain regulators mandate the continuous monitoring of a software PRNG's output to identify any lack of randomness in the results. This is necessary to prevent the state vector of the RNG algorithm from reaching an unusual value that causes it to cycle through a shorter, non-random sequence indefinitely or for an extended period. To ensure the RNG is functioning properly, performing a chi-square test on the PRNG's output and ensuring that the resulting chi-square sum remains within a predetermined threshold is a reliable monitoring method. The chi-square test is called after every RNG call and also every 10 minutes.

Background cycling

Certain regulators mandate the continuous generation of random numbers in the background, even when the application or system is not actively requesting random numbers. This process ensures a constant stream of unpredictable and statistically random values, which is crucial for various applications such as cryptography, simulations, and gaming. The process is triggered every 10 minutes and it calls `rng` random number of times (between 1 and 100).

Random numbers

RNG returns `float` in the range of `[0, 1)`.

Conversion from integer to float and back to integer are reversible operations in JavaScript and don't cause losing precision. Below code proves it by iterating over all 32 bit integers converting them to float and back to integer:

```
for (let i = 0; i < Math.pow(2, 32); i++) {  
  const power = Math.pow(2, -32);  
  const float = i * power;  
  const int = float / power;  
  if (int !== i) {  
    throw new Error("WRONG");  
  }  
}
```

NPM package

RNG is published as private `@slotify/rng` package. To have access to it you need to request private key. One of the ways to configure it is to create `.npmrc` file with the following content where `XXX` should be replaced with private key.

```
//npm.pkg.github.com/:_authToken=XXX  
@slotify:registry=https://npm.pkg.github.com/
```

After configuring token you can install `npm i @slotify/rng` and start using it in the code:

```
import * as rng from "@slotify/rng";  
  
rng.random(); //returns float in a range of [0, 1)  
  
rng.randomInteger(5); //returns integer in a range of [0, 5)
```

Docker container / RNG service

Environemnt

RNG can be run as a service inside an environment and available from games service under:

```
http://RNG_SERVICE_HOST:RNG_SERVICE_PORT
```

where `RNG_SERVICE_HOST` and `RNG_SERVICE_PORT` are passed via environment variables.

Local development

For local development the service can be downloaded from:

```
docker -p 8080:8080 run ghcr.io/slotify/rng:v1.0.0
```

and then you can request numbers from: `http://localhost:8080`

API

Path: `/api/numbers` **Method:** GET **Authorization:** NO

Generates random numbers

Query Parameters

Name	Type		Example	Description
game	string	REQUIRED	myGame	Game
provider	string	REQUIRED	myRprovider	Provider
total	string	REQUIRED	10	Total random numbers to generate

Response

Name	Type	Example	Description
numbers	number[]	[0.32050335104577243, 0.1935887192375958, 0.44593465980142355]	List of random numbers

CLI

You can generate random numbers using CLI tool. It requires `node v12` or `later` to be installed.

Running the following command:

```
npx @slotify/rng 3 #where 3 is total numbers to generate
```

will generate 3 comma separated random numbers

```
0.32050335104577243,0.1935887192375958,0.44593465980142355
```

Numbers caching and order

It is inefficient to make a call every random number is needed so you should download larger batch of numbers and cache them internally.

Numbers should be consumed from first element to last so `0.32050335104577243` first.